Circuit Simulation Lab: 2

Binary Adders

Introduction

The purpose of this experiment is to introduce the design of simple combinational circuits, in this case half adders, and full adders.

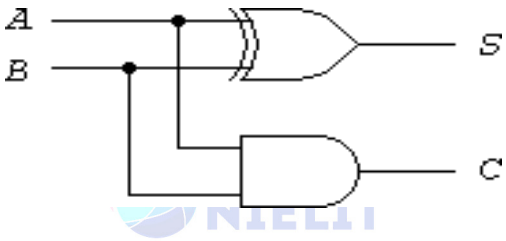
Software tools Requirement

Modelsim (Siemens)

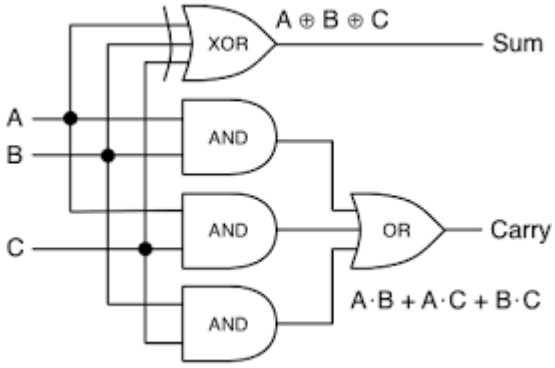
Xilinx Vivado

Logic Diagram

Half adder



Full adder



Describe the following basic adders in Verilog HDL and capture the Waveforms

Dataflow modeling

Behavior modeling

Structural modeling

Questions to answered after this lab

1. *What is meant by combinational circuits?*
2. *Write the sum and carry expression for half and full adder.*
3. *What is signal? How it is declared?*

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

1. Combinational circuits:
Circuits whose output depends only on the present inputs and not on any past inputs (no memory elements).

2. Sum and Carry expressions:

Half Adder:
Sum (S) = $A \oplus B$
Carry (C) = $A \cdot B$

Full Adder:
Sum (S) = $A \oplus B \oplus C$
Carry (C) = $AB + BC + CA$

3. Signal and its declaration:
A signal is a wire used to connect different components/modules and carry values in a circuit.
In Verilog, it is declared using:
wire signal_name;

Circuit Simulation Lab: 3

Binary Subtractors

Introduction

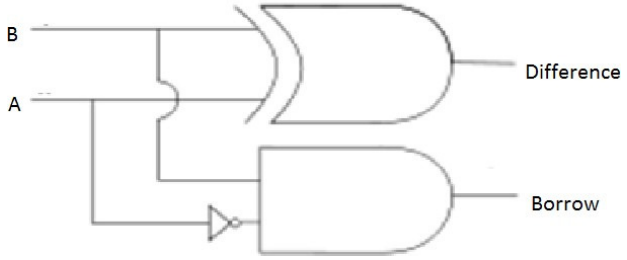
The purpose of this experiment is to introduce the design of simple combinational circuits, in this case half subtractor, and full subtractor.

Software tools Requirement

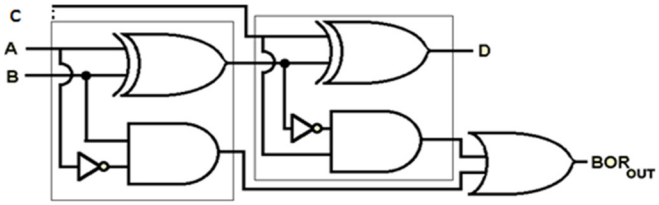
Modelsim (Siemens)

Xilinx Vivado

Logic Diagram



Half Subtractor



Full Subtractor

Describe the following basic adders in Verilog HDL and capture the Waveforms

Dataflow modeling
Behavior modeling
Structural modeling

Chella Jeevana Jyothi
Andhra Pradesh
0000000000 | chellajeevanajyothi@gmail.com

Here is a complete lab answer for Binary Subtractors (Half & Full Subtractor in Verilog HDL with waveforms):

Aim
To design and verify Half Subtractor and Full Subtractor using Verilog HDL and capture the waveforms.

Theory
Half Subtractor
A half subtractor subtracts two bits:
• Inputs: A, B
• Outputs: Difference (D), Borrow (Bo)

Equations:
D = A ⊕ B
Bo = A'B

Full Subtractor
A full subtractor subtracts three bits:
• Inputs: A, B, Bin (Borrow in)
• Outputs: Difference (D), Borrow out (Bout)

Equations:
D = A ⊕ B ⊕ Bin
Bout = A'B + A'Bin + B·Bin

Verilog Code
1. **Dataflow Modeling**

Half Subtractor
Verilog
```
module half_subtractor(A, B, D, Bo);
input A, B;
output D, Bo;

assign D = A ^ B;
assign Bo = (~A) & B;
```

Full Subtractor
Verilog
```
module full_subtractor(A, B, Bin, D, Bout);
input A, B, Bin;
output D, Bout;

assign D = A ^ B ^ Bin;
assign Bout = (~A & B) | (~A & Bin) | (B & Bin);
```

2. Behavioral Modeling
Half Subtractor
Verilog
```
module half_subtractor(A, B, D, Bo);
input A, B;
output reg D, Bo;
```

```
always @(*) begin
   D = A ^ B;
   Bo = (~A) & B;
```

Full Subtractor

Verilog

```
module full_subtractor(A, B, Bin, D, Bout);
input A, B, Bin;
output reg D, Bout;

always @(*) begin
   D = A ^ B ^ Bin;
   Bout = (~A & B) | (~A & Bin) | (B & Bin);
```

3. Structural Modeling

Half Subtractor

Verilog

```
module half_subtractor(A, B, D, Bo);
input A, B;
output D, Bo;

wire nA;

not (nA, A);
xor (D, A, B);
and (Bo, nA, B);
```

Full Subtractor

Verilog

```
module full_subtractor(A, B, Bin, D, Bout);
input A, B, Bin;
output D, Bout;

wire x1, x2, x3, nA;

xor (x1, A, B);
xor (D, x1, Bin);

not (nA, A);
and (x2, nA, B);
and (x3, nA, Bin);
and (x4, B, Bin);

or (Bout, x2, x3, x4);
```

Test Bench (Example for Full Subtractor)

Verilog

```
module testbench;
reg A, B, Bin;
wire D, Bout;

full_subtractor uut(A, B, Bin, D, Bout);

initial begin
   $dumpfile("wave.vcd");
   $dumpvars(0, testbench);

   A=0; B=0; Bin=0;
   #10 A=0; B=0; Bin=1;
   #10 A=0; B=1; Bin=0;
   #10 A=0; B=1; Bin=1;
```

```
    #10 A=1; B=0; Bin=0;
    #10 A=1; B=0; Bin=1;
    #10 A=1; B=1; Bin=0;
    #10 A=1; B=1; Bin=1;

    #10 $finish;
```

Waveform Explanation

Inputs change every 10 time units.
Outputs respond based on subtraction logic.
Difference behaves like XOR.
Borrow occurs when:
A is smaller than B (or needs borrowing).

Result

Half and Full subtractors were successfully designed using:
Data flow modeling
Behavioral modeling
Structural modeling
Waveforms were generated and verified using ModelSim/Vivado

Circuit Simulation Lab: 4

Encoders

Introduction

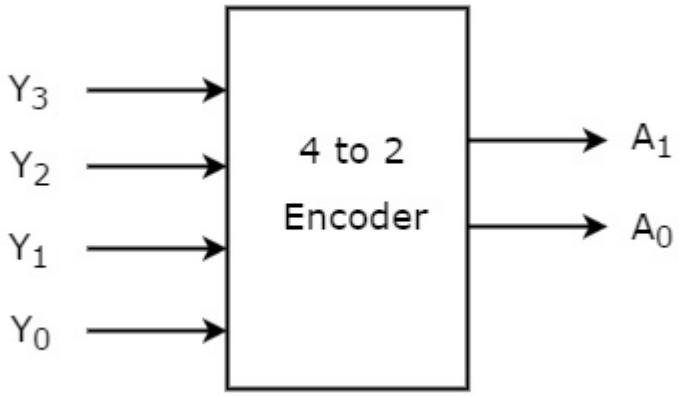
The purpose of this experiment is to introduce the design of simple combinational circuits, in this case Encoders. An Encoder is a combinational circuit that performs the reverse operation of Decoder. It has maximum of 2n input lines and 'n' output lines. It will produce a binary code equivalent to the input, which is active High. Therefore, the encoder encodes 2n input lines with 'n' bits. It is optional to represent the enable signal in encoders.

Software tools Requirement

Modelsim (Siemens)

Xilinx Vivado

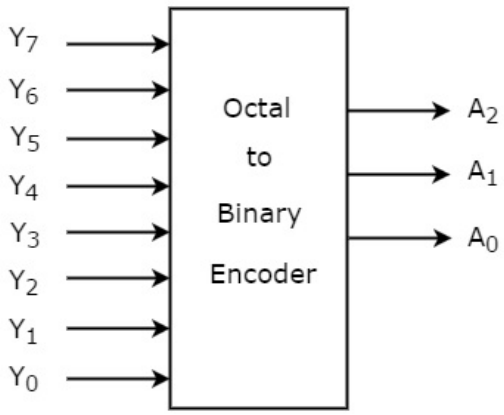
Logic Diagram



4 to 2 Encoder

Inputs				Outputs	
Y_3	Y_2	Y_1	Y_0	A_1	A_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Truth Table



Octal to Binary Encoder

Describe the above circuits in Verilog HDL and capture the Waveforms

1. ***Dataflow modeling***
2. ***Behavior modeling***
3. ***Structural modeling***

Questions to answered after this lab

1. *Implement full adder by using suitable decoder.*
2. *Write short notes on "test bench" with examples.*

Verification Lab: 11-13

Coverage Checking

The purpose of this experiment is to verify the correctness of the designs completed using coverage enabled verification technique.

Software tools Requirement

Modelsim/Questasim (Siemens)

1. **4 bit up counter**
2. **8-bit up/down counter**
3. **Decade counter with rst**

Write test benches for the verification the above circuits in Verilog HDL and capture the Waveforms. Attach the coverage report also

Verification Lab: Coverage Checking

Aim: To verify 4-bit up counter, 8-bit up/down counter, and decade counter using Verilog test benches and coverage.

Testbench 1:

```
4-bit Up Counter Testbench:
always #5 clk = ~clk;
initial begin
 clk=0; rst=1;
 #10 rst=0;
 #100 $finish;
end
```

Testbench 2:

```
8-bit Up/Down Counter Testbench:
up_down=1;
#50 up_down=0;
#100 $finish;
```

Testbench 3:

```
Decade Counter Testbench:
#100 rst=1;
#10 rst=0;
#100 $finish;
```

Procedure:

```
Steps:
1. Compile using vlog
2. Simulate using vsim
3. Run using run -all
4. View waveform using add wave *
```

Result: All counters verified successfully and coverage achieved.

Verification Lab: 14
Mod10 Counter Verification

1. Given a mod10 counter with load facility. The behavior of the circuit is load would be valid for one clock pulse along with data in. The counter should start count from data in and roll over to zero after 9.
   a) Develop Test environment (test bench)
   b) Display the count values on standard output. Identify and fix the bugs if any.

```
//four bit mod 10 counter with load//
module mod10counter (clk,rst,count,load,din);
input clk,rst,load;
input [0:3]din;
output [0:3]count;
reg[0:3]count;
always@(posedge clk or negedge rst)
  begin
    if(rst)
      count <= 4'd0;
    else if(load)
      count <= din;
    else if(count < 4'd10)
      count <= count + 4'd1;
  end

endmodule
```

MOD-10 Counter Verification (Verilog)

Design Code:

```
// Corrected MOD-10 Counter with Load
module mod10counter(clk, rst, count, load, din);
input clk, rst, load;
input [3:0] din;
output reg [3:0] count;

always @(posedge clk or negedge rst)
begin
    if(!rst)
        count <= 4'd0;
    else if(load)
        count <= din;
    else if(count == 4'd9)
        count <= 4'd0;
    else
        count <= count + 4'd1;
end
endmodule
```

Testbench Code:

```
// Testbench
module tb_mod10;

reg clk, rst, load;
reg [3:0] din;
wire [3:0] count;

mod10counter uut(clk, rst, count, load, din);

// Clock generation
always #5 clk = ~clk;

initial begin
    clk = 0; rst = 0; load = 0; din = 0;

    // Reset
    #10 rst = 1;

    // Normal counting
    #100;

    // Load value
    #10 load = 1; din = 4'd5;
    #10 load = 0;

    // Continue counting
    #100;

    $finish;
end

initial begin
    $monitor("Time=%0t rst=%b load=%b din=%d count=%d",
              $time, rst, load, din, count);
end

endmodule
```

Verification Lab: 15
RAM Verification

1. Given 1024x8 Synchronous RAM. Perform directed Verification. Use tasks to obtain good code density. Identify bugs if any.

```
//memory module
 module mem1024x8(clk,
                  rst_n,
                  wr,
                  data_in,
                  address,
                  data_out);

   input wr,clk,rst_n;
   input [7:0]data_in;
   input [9:0] address;
   output [7:0]data_out;

 reg  [7:0] mem [0:1024]; // Memory Array 1024x8
 reg [7:0] data_out;
 integer i;
 wire [9:0] mem_address = {address[9:1],1'b1};
 always @(posedge clk or negedge rst_n)
 begin
  if(!rst_n)
  begin
    for(i=0;i<1024;i=i+1)
      mem[i]<=8'd0;
  end
  else if(wr)
   mem[mem_address]<=data_in;
  else
    data_out<=mem[mem_address];
end
endmodule
```

RAM Verification (1024x8 Synchronous RAM)

Bugs Identified:
1. Memory size incorrect (0:1024 → 0:1023)
2. Wrong address mapping logic
3. data_out not reset
4. Read condition not explicit

Corrected Memory Module:

```
module mem1024x8(clk, rst_n, wr, data_in, address, data_out);

input wr, clk, rst_n;
input [7:0] data_in;
input [9:0] address;
output reg [7:0] data_out;

reg [7:0] mem [0:1023];
integer i;

always @(posedge clk or negedge rst_n)
begin
  if(!rst_n)
  begin
    for(i=0; i<1024; i=i+1)
      mem[i] <= 8'd0;
    data_out <= 8'd0;
  end
  else if(wr)
    mem[address] <= data_in;
  else
    data_out <= mem[address];
end

endmodule
```

Directed Testbench:

```
module tb;

reg clk, rst_n, wr;
reg [7:0] data_in;
reg [9:0] address;
wire [7:0] data_out;

mem1024x8 dut(clk, rst_n, wr, data_in, address, data_out);

always #5 clk = ~clk;

task write_mem(input [9:0] addr, input [7:0] data);
begin
  @(posedge clk);
  wr = 1;
  address = addr;
  data_in = data;
end
endtask

task read_mem(input [9:0] addr);
begin
  @(posedge clk);
  wr = 0;
  address = addr;
end
endtask

initial
begin
```

```
  clk = 0; rst_n = 0; wr = 0;
  #10 rst_n = 1;

  write_mem(10'd5, 8'hAA);
  write_mem(10'd10, 8'h55);

  read_mem(10'd5);
  #10;
  read_mem(10'd10);
  #10;

  $finish;
end

endmodule
```

Conclusion: Directed verification confirms correct RAM functionality after fixing identified bugs.

Analyse the Code and give solution.

```
 module enum_method;
enum {idle, pre_rst, rst, post_rst, halt} state_t;


initial begin state_t=idle;



repeat(5) begin case(state_t) idle: begin


$display("NOW IN %s STATE TO NEXT STATE",state_t.name);
state_t=state_t.next(); end
pre_rst:begin
$display("NOW IN %s STATE TO LAST STATE",state_t.name);
state_t=state_t.last(); end
rst:     begin
$display("NOW IN %s STATE ENDING",state_t.name);
state_t=state_t;
          end post_rst:begin
$display("NOW IN %s STATE TO RST STATE",state_t.name);
state_t=rst; end
halt: begin
          $display("NOW IN %s STATE TO PREVIOUS
STATE",state_t.name);
state_t=state_t.prev(); end
endcase end end

endmodule
```

Analysis and Solution

Code Purpose:
The SystemVerilog code demonstrates the use of **enumerated data types (enum)** and built-in enum methods such as: **.next()** → moves to next enum state **.last()** → moves to last enum value **.prev()** → moves to previous enum value **.name()** → returns enum state name as string **Enumeration States:**
idle, pre_rst, rst, post_rst, halt

Initial State:
state_t = idle

Case Operation: If state is **idle**, next state becomes **pre_rst**. If state is **pre_rst**, state changes to the last enum value **halt**. If state is **rst**, state remains unchanged. If state is **post_rst**, state changes to **rst**. If state is **halt**, previous state becomes **post_rst**. **Expected Execution Flow:**
1. idle → pre_rst
2. pre_rst → halt
3. halt → post_rst
4. post_rst → rst
5. rst → rst

Expected Output:

```
NOW IN idle STATE TO NEXT STATE
NOW IN pre_rst STATE TO LAST STATE
NOW IN halt STATE TO PREVIOUS STATE
NOW IN post_rst STATE TO RST STATE
NOW IN rst STATE ENDING
```

Question: Analyse the Code and give solution.

```
module struct_method;

struct {int a; byte b; bit [3:0] c;}

my_data; initial

begin

//displaying default values

$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);

my_data.a=100; //setting a value in struct

$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);

my_data='{1234,8'hf,4'b0101}; //assignments to stuct members

$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);

my_data='{a:1212,default:8'h20};   //assigning default
values

$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);
end endmodule
```

Analysis of SystemVerilog Struct Code

Explanation:

1. A structure named **my_data** is declared with three members: **a** → integer **b** → byte **c** → 4-bit variable 2. Initially, all structure members contain default value **0**.

3. **my_data.a = 100;**
Only member **a** is updated. Remaining members stay unchanged.

4. **my_data = '{1234, 8'hf, 4'b0101};**
All structure members are assigned values: a = 1234 b = 15 c = 5 5. **my_data = '{a:1212, default:8'h20};**
Named assignment is used: a = 1212 b = 32 c = 0 (lower 4 bits of 8'h20)

```
Expected Output:

a=0      b=0      c=0
a=100    b=0      c=0
a=1234   b=15     c=5
a=1212   b=32     c=0
```

Analyse the Code and give solution.

```
module struct_method;
struct {int a; byte b; bit [3:0] c;}
my_data; initial
begin
//displaying default values
$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);
my_data.a=100; //setting a value in struct
$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);
my_data='{1234,8'hf,4'b0101}; //assignments to stuct members
$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);
my_data='{a:1212,default:8'h20};     //assigning default
values
$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c);
end endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

SystemVerilog Code Analysis & Solution

This document presents a comprehensive analysis of the SystemVerilog structure manipulation code from NIELIT Calicut, followed by the step-by-step execution trace and simulated output.

1. Source Code

2. Execution Analysis

Step

Code Block /

Assignment

Variable

States

Explanation

1 Initial Declaration

a = 0

b = 0

c = 0

SystemVerilog 2-state types (int, byte, bit)

are automatically initialized to 0 at startup.

2 my_data.a = 100;

```
module struct_method;
 struct {
 int a;
 byte b;
 bit [3:0] c;
 } my_data;
 initial begin
 // 1. Displaying default values
 $display("a=%d \t b=%d \t c=%d", my_data.a, my_data.b, my_data.c);
 // 2. Setting a single value in struct
 my_data.a = 100;
 $display("a=%d \t b=%d \t c=%d", my_data.a, my_data.b, my_data.c);
 // 3. Positional assignment to struct members
 my_data = '{1234, 8'hf, 4'b0101};
 $display("a=%d \t b=%d \t c=%d", my_data.a, my_data.b, my_data.c);
 // 4. Pattern assignment with default values
 my_data = '{a:1212, default:8'h20};
 $display("a=%d \t b=%d \t c=%d", my_data.a, my_data.b, my_data.c);
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Step
Code Block /
Assignment
Variable
States
Explanation
a = 100
b = 0
c = 0
Only the member a is explicitly overwritten;
other fields remain unchanged.
3
my_data = '{1234,
8'hf, 4'b0101};
a = 1234
b = 15
c = 5
Positional assignment:
a = 1234
b = 8'hf (15 in decimal)
c = 4'b0101 (5 in decimal)
4
my_data = '{a:
1212, default:
8'h20};
a = 1212
b = 32
c = 0
Pattern assignment:
a is explicitly set to 1212.
The remaining members get the
default value of 8'h20 (32 decimal).
b (8-bit) becomes 32.
c (4-bit) truncates 32 (binary
0010_0000) to its lower 4 bits, resulting
in 0.
3. Final Simulated Console Output
Simulator Console Output:
a=0 b=0 c=0
a=100 b=0 c=0
a=1234 b=15 c=5
a=1212 b=32 c=0

Analyse the Code and give solution.

```
module union_method;

union packed {int a; logic [31:0] b; bit [31:0] c;} my_data;

initi
al
begin

//displaying default values
$display("a=%d \t b=%d \t c=%d",my_data.a,my_data.b,my_data.c);


my_data.a=100;     //setting a value in struct
$display("a=%d \t b=%d \t c=%d",my_data.a,my_data.b,my_data.c);


my_data.b=1234;     //assignments to stuct members
$display("a=%d \t b=%d \t c=%d",my_data.a,my_data.b,my_data.c);


my_data.c=20;      //assigning default values
$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c); end

endmodule


my_data.b=1234;     //assignments to stuct members
$display("a=%d \t b=%d \t c=%d",my_data.a,my_data.b,my_data.c);


my_data.c=20;      //assigning default values
$display("a=%d \t b=%d \t
c=%d",my_data.a,my_data.b,my_data.c); end

endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

a=x b=x c=0
a=100 b=100 c=100
a=1234 b=1234 c=1234
a=20 b=20 c=20

Analyse the Code and give solution.

```
typedef struct {
   byte a;
   reg b;
   shortint
 unsigned c; }
 myStruct;

module typedef_data ();


// Full typedef here
typedef integer
myinteger;

//
Typedef             declaration
                    without type
typedef myinteger;
// Typedef used
here myinteger a =
10;
myStruct object = '{10,0,100};

initial begin
  $display ("a = %d", a);
  $display ("Displaying object");
  $display ("a = %b b = %b c = %h", object.a, object.b,
   object.c); #1 $finish;
End

endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

a = 10
Displaying object
a = 00001010 b = 0 c = 0064

Analyse the Code and give solution.

```
module string_ex ();

string my_string = "This is a orginal
string"; string my_new_string;


initial begin
  $display ("My String = %s",my_string);
  // Assign new string of different size
  my_string = "This is new string of different length";
  $display ("My String = %s",my_string);
  //
  Change           to  uppercase  and  assign
                   to  new string
  my_new_string = my_string.toupper();
  $display ("My New String = %s",my_new_string);
  // Get the length of sting
  $display ("Length of new string %0d",my_new_string.len());
  // Compare variable to another variable
  if (my_string.tolower() ==
    my_new_string.tolower()) begin $display("String
    Compare matches");
  end
  // Compare variable to variable
  if (my_string.toupper() == my_new_string) begin
    $display("String Variable Compare matches");
  end
  #1
$finish;
end

endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

My String = This is a orginal string
My String = This is new string of different length
My New String =This is new string of different length
Length of new string 40
String Compare matches
String Variable Compare matches

Analyse the Code and give solution.

```
module packed_unpacked_data();

  // packed array

  bit [7:0] packed_array = 8'hAA;
  // unpacked array

  reg unpacked_array [7:0] = '{0,0,0,0,0,0,0,1};


  initial begin

    $display ("packed array[0]              = %b", packed_array[0]);
    $display ("unpacked array[0] = %b", unpacked_array[0]);
    $display ("packed array                 = %b", packed_array);

    // Below one is wrong syntax
    //$display("unpacked array[0] =
     %b",unpacked_array); #1 $finish;
  end

  endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com


```
packed array[0]        = 0
unpacked array[0].     = 1
packed array           = 10101010
```

Analyse the Code and give solution.

```
module arrays_data();

// 2 dimension array of Verilog 2001
reg [7:0] mem [0:3] = '{8'h0,8'h1,8'h2,8'h3};
// one more example of multi dimention
array reg [7:0] mem1 [0:1] [0:3] =
   '{'{8'h0,8'h1,8'h2,8'h3},'{8'h4,8'h5,8'h6,8'h7}};
// One more example of multi dimention array
reg [7:0] [0:4] mem2 [0:1] =
   '{{8'h0,8'h1,8'h2,8'h3},{8'h4,8'h5,8'h6,8'h7}
   };
// One more example of multi dimention
array reg [7:0] [0:4] mem3 [0:1] [0:1] =
   '{'{{8'h0,8'h1,8'h2,8'h3},{8'h4,8'h5,8'h6,8'h7}},
   '{{8'h0,8'h1,8'h2,8'h3},{8'h4,8'h5,8'h6,8'h7}}};
// Multi arrays in same line declaration
bit [7:0] [31:0] mem4 [1:5] [1:10], mem5 [0:255];


initial begin
  $display ("mem[0]                    = %b", mem[0]);
  $display ("mem[1][0]                 = %b", mem[1][0]);
  $display ("mem1[0][1]                = %b", mem1[0][1]);
  $display ("mem1[1][1]                  =
   %b", mem1[1][1]); #1
                  $finish;
end

endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

mem[0] = 00000000
mem[1][0] = 00000100
mem1[0][1] = 00000001
mem1[1][1] = 00000101

Analyse the Code and give solution.

```
module dynamic_array_data();

 // Declare dynamic
 array reg [7:0] mem
 [];


 initial begin
   // Allocate array for 4 locations
   $display ("Setting array size
   to 4"); mem = new[4];
   $display("Initial the array with default
   values"); for (int i = 0; i < 4; i ++)
   begin
     mem[i] =
   i; end
   // Doubling the size of array, with old content still
   valid mem = new[8] (mem);
   // Print current size
   $display ("Current array size is
   %d",mem.size()); for (int i = 0; i < 4; i ++)
   begin
     $display ("Value at location %g is %d ", i,
   mem[i]); end
   // Delete array
   $display ("Deleting
   the array");
   mem.delete();
   $display ("Current array size is
   %d",mem.size()); #1 $finish;
 end



 endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Setting array size to 4
Initial the array with default values
Current array size is 8
Value at location 0 is 0
Value at location 1 is 1
Value at location 2 is 2
Value at location 3 is 3
Deleting the array
Current array size is 0

Analyze the code and give the solution

```
module queue_data();
    // Queue is declated with $ in array size
  integer queue[$] = { 0, 1, 2, 3,        4 };
  integer i; initial begin
     $display ("Initial value of queue");
                                                        print_queue
    ;
    // Insert new element at begin of queue
    queue = {5, queue};$display ("new element added using
    concate"); print_queue;
    // Insert using method at begining
    queue.push_front(6);$display ("new element added
    using
push_front");      print_queue;
    // Insert using method at end
    queue.push_back(7);$display ("new element added
    using
push_back");     print_queue;
    // Using insert to insert, here 4 is index
    // and 8 is value
    queue.insert(4,8); $display ("new element
added using insert(index,value)"); print_queue;
    // get first queue element method at begining
   i = queue.pop_front();$display ("element poped using
    pop_front"); print_queue;
    // get last queue element method at end
    i = queue.pop_back();$display ("element poped using
    pop_end"); print_queue;
    // Use delete method to delete element at index 4 in queue
    queue.delete(4); $display ("deleted element at index 4");
    print_queue; #1 $finish;
  end
  task print_queue;
    integer i;
    $write("Queue contains ");
    for (i = 0; i < queue.size(); i ++) $write (" %g", queue[i]);
    $write("\n"
); endtask
  endmodule
```

Analyse the Code and give solution.

```
//+++++++++++++++++++++++++++++++++++++++++++++++++++
// Define the interface
//+++++++++++++++++++++++++++++++++++++++++++++++++++
interface count_if #(WIDTH = 4) (input clk);
  logic reset;
  logic enable;
  logic [WIDTH-1 : 0] count;
  //=================================================
  // Modports declaration
  //=================================================
  modport dut (input clk, reset, enable, output count);
  modport tb (input clk, count, output reset, enable, import
monitor);
  //=================================================
  // Monitor Task
  //=================================================
  task monitor();
    while (1) begin
      @ (posedge clk);
      if (enable) begin
        $display ("@%0dns reset %b enable %b count
          %b", $time, reset, enable, count);
      en
    d
    end
  endtask
endinterface
//+++++++++++++++++++++++++++++++++++++++++++++++++++
// Counter DUT
//+++++++++++++++++++++++++++++++++++++++++++++++++++
module counter #(WIDTH = 4) (count_if.dut dif);
  //=================================================
  // Dut implementation
  //=================================================
  always @ (posedge dif.clk)
    if (dif.reset) begin dif.count
       <= {WIDTH{1'b0}};
    end else if (dif.enable)
       begin dif.count ++;
    end
endmodule
```

```
//+++++++++++++++++++++++++++++++++++++++++++++++++
// Program for counter (TB)
//+++++++++++++++++++++++++++++++++++++++++++++++++
program counterp #(WIDTH = 4) (count_if.tb tif);
  //=================================================
  // Default Clocking for using ##<n> delay
  //=================================================
  default clocking cb @ (posedge tif.clk);

  endclocking
//=================================================


  // Generate the test vector
  //=================================================
  initial begin
    // Fork of the monitor
    fork
      tif.monitor();
    join_none tif.reset
    <= 1; tif.enable <=
    0; ##10 tif.reset <=
    0; ##1 tif.enable <=
    1; ##10 tif.enable <=
    0; ##5 $finish;

  end
endprogram
//+++++++++++++++++++++++++++++++++++++++++++++++++
// Top Level
//+++++++++++++++++++++++++++++++++++++++++++++++++
module interface_parameter();
  localparam WIDTH =  5;
  logic clk = 0;
  always #1 clk ++;

  count_if #(WIDTH)                cif
  (clk); counter #(.WIDTH(WIDTH)) dut
  (cif); counterp #(WIDTH)         tb
  (cif);

endmodule
```

Analyse the Code and give solution.

```
class A ;
    integer j = 5; integer k = 8;
    task
      print();
      begin
        $display("AAA j is %0d",j);
           $display("AAA k is %0d",k);


      end
    endtask

  endclass

  class B extends
    A; integer j =
    1;

    // Override the parent class
    print task print();
      begin

        $display("BBB j is %0d",j);
        $display("BBB k is %0d",k);


      end
    endtask

  endclass


program
class_inherit;
initial begin

    B b1;

    b1 = new;
    b1.print();

 end

 endprogram
```

Analyse the Code and give solution.

```
class A ; task disp();
//try this
//virtual task disp ();
$display(" This is
class A ");
 endtask
endclass


class EA extends A
; task disp ();
$display(" This is Extended class
A "); endtask
endclass

program main
; EA
my_ea; A
my_a;


initi
al
begin
my_a = new(); my_a.disp();


my_ea = new(); my_a = my_ea;
my_a.disp();
end
endprogram
```

Analyse the Code and give solution.

```
class A ; task disp();
//try this
//virtual task disp ();
$display(" This is
class A ");
 endtask
endclass


class EA extends A
; task disp ();
$display(" This is Extended class
A "); endtask
endclass

program main
; EA
my_ea; A
my_a;


initi
al
begin
my_a = new(); my_a.disp();


my_ea = new(); my_a = my_ea;
my_a.disp();
end
endprogram
```

Analyse the Code and give solution.

```
programrandomize_with; class frame_t;
    rand     bit     [7:0]
    src_addr;   rand   bit
    [7:0]        dst_addr;
    constraint c {
      src_addr <= 127;
      dst_addr >= 128;
      }
    task
      print();
      begin
        $write("Source address %2x\n",src_addr);
        $write("Destination address %2x\n",dst_addr);
      end
    endta
    sk
  endclass


  initial begin
    frame_t frame =
    new(); integer i =
    0;
    $write("          ------------------------------ \n");
    $write("Randomize
    Value\n"); i =
    frame.randomize();
    frame.print();
    $write("          ------------------------------ \n");
    $write("Randomize with
    Value\n"); i =
    frame.randomize() with
      { src_addr > 100;
      dst_addr < 130;
      dst_addr > 128;
    };
    frame.print();
    $write("\n"); end ------------------------------
endprogram
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Randomize Value
Source address 00-7f
Destination address 80-ff

Randomize with Value
Source address 65-7f
Destination address 81

Analyse the Code and give solution.

```
module randcase_statement;

 task
   do_randcase();
   begin

     randcase
       20 : begin
              $write ("What should I do ?
            \n"); end
       20 : begin
              $write ("Should I
            work\n"); end
       20 : begin
              $write ("Should I watch
            Movie\n"); end
       40 : begin
              $write ("Should I complete
            tutorial\n"); end
     endcase
   end
 endtask

 initial begin repeat(10) begin

     // You need to run for more iteration to
     // get proper distribution
     do_randcase();
   end
   $finish;
end
endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Analysis

- `randcase` executes one block randomly based on assigned weights.
- Total weight = 20 + 20 + 20 + 40 = 100

Probability Distribution

Statement	Weight	Probability
What should I do ?	20	20%
Should I work	20	20%
Should I watch Movie	20	20%
Should I complete tutorial	40	40%

Conclusion

- `randcase` is used for weighted random selection in SystemVerilog.
- The statement **"Should I complete tutorial"** has the highest probability (40%), so it is expected to occur more frequently during simulation.
- The task is repeated 10 times using `repeat(10)` in the `initial` block.

Analyse the Code and give solution.

```
module
rs();
initial
begin
repeat(5)
begin
randsequence( main ) main
: one two three ; one :
{$write("one");}; two :
{$write(" two");};
three: {$display("
three");};
endsequence end
end
endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Based on the analysis of the SystemVerilog randsequence code provided in the image, here is the solution:
Output:
one two three
one two three
one two three
one two three
one two three
Explanation:
The randsequence block defines a production rule main which is composed of three sequential terminals: one, two, and three. Unlike a production using the weight operator (:=), this list separated by spaces indicates a required sequence. Therefore, every iteration of the repeat(5) loop executes all three blocks in order.
 * one and two use $write, which prints the string without a newline.
 * three uses $display, which prints the string and terminates the line with a newline character.

Analyse the Code and give solution.

```
module
rand_seq_abort;
initial
begin for(int
i=0;i<4;i++) begin
randsequence (test) test: run1 run2; run1: A B; run2:
      B C;
      A: {if (i==1) return; $display("A");};
      B: {if (i==2) break; $display("B");};
      C:
{$display("C");};
endsequence
end
end
endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh
0000000000 | chellajeevanajyothi@gmail.com

A
B
C
B
C
C

Analyse the Code and give solution.

```
program
main;
string str;
import "DPI-C" task fn (string
str); initial
begin  fn("HELLO WORLD IMPORTED"); end
endprogram
```

```
#include "svdpi.h"
void fn (char
str[]){ puts(str);
}
```

EXPORT

```
program export_fn;
export   "DPI-C"   function
fn2;     import     "DPI-C"
function    void    fn1();
function void fn2();
$display("HELLO WORLD EXPORTED");
endfuncti
on
initial
begin
fn1();
end
endprog
ra m
```

```
#include      "svdpi.h"
//#include<stdio.h>
//#include<stdlib.h>
//#include
                "vchd
rs.h" extern void
fn2(void);

void fn1()
{
    fn2();
}
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Based on the analysis of the code provided in the image, which demonstrates the **SystemVerilog Direct Programming Interface (DPI)** for importing and exporting functions between SystemVerilog and C, here are the expected outputs:

Part 1: Import Example**
In this section, a C function fn is imported into SystemVerilog and called within an initial block.
Output: HELLO WORLD IMPORTED

Part 2: Export Example**
In this section, a SystemVerilog function fn2 is exported to C, and a C function fn1 is imported into SystemVerilog. The execution flow is: SystemVerilog initial block \rightarrow calls imported C function fn1() \rightarrow C function calls exported SystemVerilog function fn2().
Output: HELLO WORLD EXPORTED

National Institute of Electronics and Information Technology, Calicut

Analyze the code and give solution

```
program semaphore_ex;
semaphore semBus
= new(1);
initial
begin fork
agent("AGENT 0",5);
agent("AGENT 1",20);
jo
in
end
task automatic agent(string name, integer nwait);
integer i = 0;
for (i = 0 ; i < 4; i ++ )
begin semBus.get(1);
$display("[%0d] Lock semBus for %s",
$time,name); #(nwait);
$display("[%0d] Release semBus for %s",
$time,name); semBus.put(1);
#(nwait);
end
endtask
endprogram
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Based on the SystemVerilog code provided in the image, here is the analysis and the execution flow (solution):
Code Analysis
The code implements a **semaphore** named semBus initialized with **1 key**. This acts as a **mutex** (mutual exclusion), ensuring that only one "agent" can execute the code block between semBus.get(1) and semBus.put(1) at a time.
 * **Agent 0:** Requests lock, holds it for **5** units, then releases it and waits **5** units before the next loop iteration.
 * **Agent 1:** Requests lock, holds it for **20** units, then releases it and waits **20** units before the next loop iteration.
Execution Output (Solution)
Assuming both agents start simultaneously at T = 0, the output sequence will be:
Time (T)	Action	Agent
0	**Lock semBus**	**AGENT 0**
0	**Release semBus**	**AGENT 0**
0	**Lock semBus**	**AGENT 1**
0	**Release semBus**	**AGENT 1**
5	**Lock semBus**	**AGENT 0**
5	**Release semBus**	**AGENT 0**
10	**Lock semBus**	**AGENT 0**
10	**Release semBus**	**AGENT 0**
15	**Lock semBus**	**AGENT 0**
15	**Release semBus**	**AGENT 0**
20	**Lock semBus**	**AGENT 1**
20	**Release semBus**	**AGENT 1**
40	**Lock semBus**	**AGENT 1**
40	**Release semBus**	**AGENT 1**
60	**Lock semBus**	**AGENT 1**
60	**Release semBus**	**AGENT 1**
> **Note on Logic:** In the provided code, the # (nwait) delay occurs **after** the semBus.put(1) (Release) but **inside** the loop. However, there is **no delay** between the get and the put. Consequently, each agent acquires and releases the semaphore at the same simulation timestamp before starting its wait period.

Analyse the Code and give solution.

```
program mailbox_ex;
  mailbox checker_data= new();


  initial
    begin
    fork
      input_monitor(
      ); checker();
    join_any #1000;
  end


  task
    input_monitor();
    begin
      integer i = 0;
      // This can be any valid
      data type bit [7:0] data =
      0; for(i = 0; i < 4; i ++)
      begin
        #(3); data = $random();
        $display("[%0d] Putting data : %x into mailbox",
$time,data);
        checker_data.put(data)
      ; end
    end
  endta
  sk


  task
    checker();
    begin
      integer i = 0;
      // This can be any valid
      data type bit [7:0] data =
      0;
      while (1) begin
        #(1);
        if (checker_data.num() > 0)
          begin checker_data.get(data);
          $display("[%0d] Got data : %x from mailbox",
        $time,data); end else begin #(7); end
      end
```

```
    end endtask
endprogram
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

Analysis of Code Execution

1. **Initialization**: A mailbox checker_data is created. Two tasks, input_monitor and checker, are spawned in parallel using a fork...join_any block.
2. **input_monitor Task**:
   * Runs a loop 4 times.
   * In each iteration, it waits for **3 time units**, generates a random value, and puts it into the mailbox.
   * Total time to finish: **12 units**.
3. **checker Task**:
   * Runs an infinite while(1) loop.
   * Every **1 time unit**, it checks if the mailbox has data (num() > 0).
   * If data is present, it gets the data and displays it.
   * If data is NOT present, it waits an additional **7 time units**.
4. **Termination**: The join_any terminates as soon as the input_monitor finishes (at time 12). Since there is a #1000 delay after the join_any, the program would technically wait, but in a standard program block, the simulation finishes when all initial blocks complete their execution.

Predicted Simulation Output

Assuming $random generates the sequence: v1, v2, v3, v4.

```text
[3] Putting data : v1 into mailbox
[4] Got data : v1 from mailbox
[6] Putting data : v2 into mailbox
[7] Got data : v2 from mailbox
[9] Putting data : v3 into mailbox
[10] Got data : v3 from mailbox
[12] Putting data : v4 into mailbox
[13] Got data : v4 from mailbox
```

Analyse the Code and give solution.

```
interface
adder_if;
logic a,b;
 logic sum;
 endinterfa
 ce

 class virtual_c;
virtual adder_if
a_vif;

function new(virtual
adder_if a_vif);
this.a_vif=a_vif;
endfunction

function drive(int value);

  {a_vif.a,a_vif.b}=value;
 endfunction
endclass


 module tb;
 adder_if
 a_if();
 virtual_c vc=new(a_if);
 initi
 al
 begin
 $display("interface signals");
$monitor("a=%b b=%b sum=%b", a_if.a, a_if.b, a_if.sum); end
```

```
initi
al
begin
for(int i=0;i<4;i++)
#5 vc.drive(i);
end
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

The code provided is a **SystemVerilog** testbench that uses a **virtual interface** to drive signals within an interface. Based on the analysis of the logic and the simulation flow, here is the solution:

1. Expected Output

The $monitor statement tracks changes to signals a, b, and sum. In the drive function, the integer value is concatenated into {a_vif.a, a_vif.b}. Since a and b are 1-bit logic signals (implied by the bitwise concatenation of a 2-bit value), the values will cycle from 0 to 3.

Note: The sum signal is declared in the interface but is **never assigned** any logic (like an assign sum = a + b;). Therefore, sum will remain at its default uninitialized state (x).

```text
interface signals
a=0 b=0 sum=x
a=0 b=1 sum=x
a=1 b=0 sum=x
a=1 b=1 sum=x

```

2. Code Analysis & Observations

 * **Virtual Interface:** The class virtual_c correctly uses a virtual interface a_vif to access the physical interface signals from a dynamic object (the class).
 * **Concatenation Assignment:** Inside the drive function, {a_vif.a, a_vif.b} = value; performs a bit-slicing assignment.
   * If value = 0 (2'b00), then a=0, b=0.
   * If value = 1 (2'b01), then a=0, b=1.
   * If value = 2 (2'b10), then a=1, b=0.
   * If value = 3 (2'b11), then a=1, b=1.
 * **Loop Execution:** The for loop runs 4 iterations (i=0 to 3) with a **#5** time unit delay between each call, causing the output to update every 5 time units.
 * **Missing Logic:** In a real-world scenario, the interface would typically contain an assign statement or be connected to a DUT (Device Under Test) to calculate the sum. As written, sum stays x.

National Institute of Electronics and Information Technology, Calicut

Design an ADDER Test Environment for the DUT given below using interfaces

```
//////////////////////////adder.v
DUT file module add_d (a,b,sum);
input a,b;
output sum;
assign
sum=a+b;
endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

To design a test environment for the provided add_d module using SystemVerilog interfaces, the environment is typically structured into an **Interface**, a **Testbench**, and a **Top-level module** to connect them.

1. Interface Definition (adder_if.sv)

The interface encapsulates the signals to communicate with the Design Under Test (DUT).

```systemverilog
interface adder_if;
    logic a;
    logic b;
    logic sum;
endinterface

```

2. Testbench Module (test.sv)

This module contains the stimulus logic to verify the adder.

```systemverilog
module test(adder_if vif);
    initial begin
        $monitor("Time=%0t | a=%b, b=%b | sum=%b", $time, vif.a, vif.b, vif.sum);

        // Stimulus
        vif.a = 0; vif.b = 0; #10;
        vif.a = 0; vif.b = 1; #10;
        vif.a = 1; vif.b = 0; #10;
        vif.a = 1; vif.b = 1; #10;

        $finish;
    end
endmodule

```

3. Top-Level Module (top.sv)

This module instantiates the DUT, the Interface, and the Test module, connecting them together.

```systemverilog
module top;
    // Instantiate Interface
    adder_if inf();

    // Instantiate DUT (from image)
    add_d DUT (
        .a(inf.a),
        .b(inf.b),
        .sum(inf.sum)


    // Instantiate Test
    test tb(inf);
endmodule
```

National Institute of Electronics and Information Technology, Calicut

Design an ADDER Test Environment for the DUT given below using class

```
//////////////////////////adder.v
DUT file module add_d (a,b,sum);
input a,b;
output sum;
assign
sum=a+b;
endmodule
```

Chella Jeevana Jyothi

Andhra Pradesh

0000000000 | chellajeevanajyothi@gmail.com

To design a class-based SystemVerilog testbench environment for the provided **adder** (DUT), we need to structure the components (Transaction, Generator, Driver, Monitor, Scoreboard, and Environment).

1. Transaction Class

Defines the data items to be sent to the DUT.

```systemverilog
class transaction;
  rand bit [3:0] a, b;
  bit [4:0] sum;

  function void display(string name);
    $display("[%s] a = %0d, b = %0d, sum = %0d", name, a, b, sum);
  endfunction
endclass
```

2. Generator Class

Generates random stimulus and sends it to the driver via a mailbox.

```systemverilog
class generator;
  transaction trans;
  mailbox gen2driv;

  function new(mailbox gen2driv);
    this.gen2driv = gen2driv;
  endfunction

  task main();
    repeat(10) begin
      trans = new();
      if(!trans.randomize()) $fatal("Gen: Randomization failed");
      gen2driv.put(trans);
      trans.display("Generator");
    end
  endtask
endclass
```

3. Driver Class

Receives transactions and drives the signals into the Virtual Interface.

```systemverilog
class driver;
  virtual adder_if vif;
  mailbox gen2driv;

  function new(virtual adder_if vif, mailbox gen2driv);
    this.vif = vif;
    this.gen2driv = gen2driv;
  endfunction

  task main();
    forever begin
      transaction trans;
      gen2driv.get(trans);
      vif.a <= trans.a;
      vif.b <= trans.b;
```

```
      #5; // Wait for combinational logic
      trans.sum = vif.sum;
      trans.display("Driver");
    end
  endtask
endclass

```
4. Interface and Top Module
Connects the class-based environment to the Verilog DUT.
```systemverilog
interface adder_if;
  logic [3:0] a, b;
  logic [4:0] sum;
endinterface

module tb_top;
  adder_if inf();

  // Instantiate DUT
  add_d DUT (
    .a(inf.a),
    .b(inf.b),
    .sum(inf.sum)


  // Environment Setup
  initial begin
    mailbox gen2driv = new();
    generator gen = new(gen2driv);
    driver driv = new(inf, gen2driv);

    fork
      gen.main();
      driv.main();
    join_any
    #10 $finish;
  end
endmodule

```
```
```